Lab5 Report

Name: 时分

Student ID: PB23151799

Purpose

The purpose is to develop a sequence detector in LC-3.

On starting, display "SD is ready! Please input your number:" on the screen. The sequence detector expects a sequence of '0' and '1' typed in by keyboard. After entering the sequence, type 'y' to submit. The finite state machine should find and count how many "1010" in the sequence. After finding out the result, display "There are(is) X 1010 in the sequence!" Where X is the number of 1010 in the input sequence.

Program Design

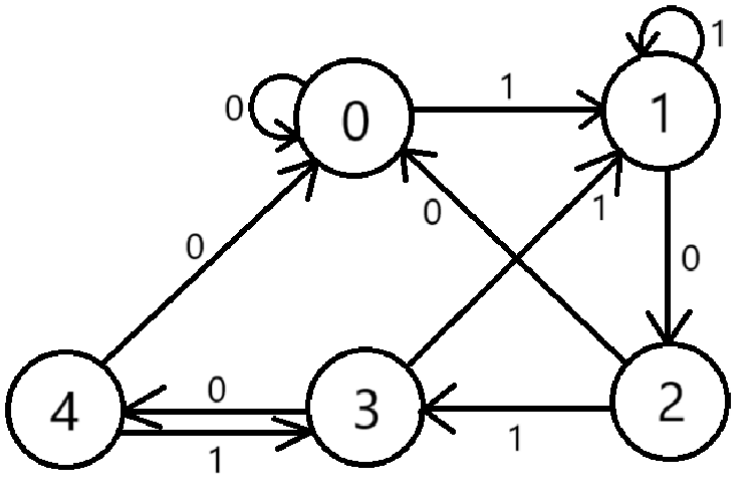
There are 4 parts in the program:

1. Display starting text.
2. Input and analyse the sequence in the finite state machine.
3. Transform the result from binary number to ASCII string.
4. Display the result.

The first part is easy to realize by using PUTS (TRAP x22) in subroutine PRINTSTART.

```
PRINTSTART
            LEA     R0, TEXT0
            PUTS
            RET
TEXT0       .STRINGZ "SD is ready!Please input your number:"
```

To analyse the sequence in part 2, we need to design the finite state machine. The state diagram is as below. The machine starts at state 0, and state 4 means there is a "1010" in the sequence.



Obviously, if we try to design branches for every input in each state, the program will be very complicated. We can notice something unique (which only happens when detecting "1010"):

1. In state 0~3:

   (1) If input + state is odd, the machine will be set to the next state.

   (2) If input + state is even, the machine will be reset to 0 when state and input are even, and to 1 when state and input are odd.

2. In state 4:

   (1) If input is '0' (input + state is even), the machine will be reset to 0.

   (2) If input is '1' (input + state is odd), the machine will be set to state 3.

So we can design the branches like:

1. input + state is odd:

   (1) state = 4: Set state to 3.

   (2) state != 4: Set state to the next.

2. input + state is even:

   Reset state to [input AND x0001].

And when the next state is 4, the counting adds by 1. In this way, we even don't have to transform input from ASCII to binary number, because the ASCII code and the binary number share the same odevity.

The assembly code is as below:

```
            JSR     PRINTSTART
            LD      R1, ASCII_LY    ;R1 = -x79 = -'y'
            AND     R2, R2, #0      ;R2 stores the state
            AND     R3, R3, #0      ;R3 counts 1010
LOOP        GETC
            OUT
            ADD     R5, R0, R1
            BRz     DONE            ;if input = 'y', goto DONE
            ADD     R5, R0, R2
            AND     R5, R5, #1
            BRz     RESET           ;if state + input is even, goto RESET
            ADD     R5, R2, #-4
            BRnp    SKIP
            ADD     R2, R2, #-2     ;if state = 4 && input = 1, R2 = 3
SKIP        ADD     R2, R2, #1      ;if state + input is odd, R2++
            BRnzp   TEST4
RESET       AND     R2, R0, #1
TEST4       ADD     R5, R2, #-4
            BRnp    NOT4            ;if state = 4, R3++
            ADD     R3, R3, #1
NOT4        BRnzp   LOOP
DONE        JSR     PRINTRESULT
            HALT
ASCII_LY    .FILL   xFF87
```

As LC-3 has 16-bit addressability, the largest positive number is $2^{15}-1=32767$ in 2's complement, so we only convert a 5-demical-digit number into an ASCII string. The program subtracts 10000 from R3 repeatingly until R3 < 0, to get the "ten thousands" digit, and test the other digits in the same way. To save memory, we can test every digit in a loop by using R5 as a counter of digit. The full assembly code of the subroutine is:

```
BINARYTOASCII
            LEA     R0, ASCIIBUFF
            LEA     R1, ASCIIBUFF    ;R1 is the pointer
            AND     R5, R5, #0       ;R5 counts the digit, R5 = 4 at first
            ADD     R5, R5, #4
LOOPA       AND     R4, R4, #0       ;R4 = -10 ^ R5
            ADD     R2, R5, #-4
            BRnp    SKIPW
            LD      R4, NEG1W
            BRnzp   STARTA
SKIPW       ADD     R2, R5, #-3
            BRnp    SKIPK
            LD      R4, NEG1K
            BRnzp   STARTA
SKIPK       ADD     R2, R5, #-2
            BRnp    SKIP100
            LD      R4, NEG100
            BRnzp   STARTA
SKIP100     ADD     R2, R5, #-1
            BRnp    SKIP10
            ADD     R4, R4, #-10
            BRnzp   STARTA
SKIP10      ADD     R4, R4, #-1      ;decide R4
STARTA      LD      R2, ASCII_0      ;R2 is the digit
LOOPB       ADD     R3, R3, R4
            BRn     ENDA
            ADD     R2, R2, #1
            BRnzp   LOOPB            ;get the digit
ENDA        STR     R2, R1, #0
            ADD     R1, R1, #1       ;store the digit and move the pointer
            NOT     R4, R4
            ADD     R4, R4, #1       ;R4 = +10 ^ R5
            ADD     R3, R3, R4       ;correct R3 for one-too-many substracts
            LD      R4, ASCII_N0
            ADD     R4, R2, R4
            BRnp    SKIPA            ;if R2 = '0' && R5 != 0, R0++
            ADD     R4, R5, #0
            BRz     SKIPA
            ADD     R0, R0, #1
SKIPA       ADD     R5, R5, #-1      ;move the digit
            BRzp    LOOPA
            RET
ASCII_N0    .FILL   xFFD0
ASCII_0     .FILL   x0030
NEG1W       .FILL   #-10000
NEG1K       .FILL   #-1000
NEG100      .FILL   #-100
ASCIIBUFF   .STRINGZ "00000"
```

The last part is also easy. Because BINARYTOASCII is called in the subroutine, we have to store R7 at first. The assembly code of the subroutine PRINTRESULT is as below:

```
PRINTRESULT
            AND     R0, R0, #0
            ADD     R0, R0, xA
            OUT                     ;print '\n'
            ST      R7, SAVE_R7
            LEA     R0, TEXT1
            PUTS                    ;print "There are(is) "
            JSR     BINARYTOASCII
            ;transform the binary number in R3 to ASCII string
            PUTS
            LEA     R0, TEXT2
            PUTS                    ;print " 1010 in the sequence!"
            LD      R7, SAVE_R7
            RET
TEXT1       .STRINGZ "There are(is) "
TEXT2       .STRINGZ " 1010 in the sequence!"
SAVE_R7     .BLKW   1
```

Testing Evidence (Screenshots)

1. Input: 1111y

   Output: There are(is) 0 1010 in the sequence!

   ```
   SD is ready!Please input your number:1111y
   There are(is) 0 1010 in the sequence!


   --- Halting the LC-3 ---
   ```

2. Input: 11**1010**00**1010**1y

   Output: There are(is) 2 1010 in the sequence!

   ```
   SD is ready!Please input your number:11101000101011y
   There are(is) 2 1010 in the sequence!


   --- Halting the LC-3 ---
   ```

3. Input: **101010**1110y

   Output: There are(is) 2 1010 in the sequence!

   ```
   SD is ready!Please input your number:1010101110y
   There are(is) 2 1010 in the sequence!


   --- Halting the LC-3 ---
   ```

4. Input: **10101010101010101010101010**y (12 * "10")

Output: There are(is) 12 1010 in the sequence!

```
SD is ready!Please input your
number:10101010101010101010101010y
There are(is) 12 1010 in the sequence!

--- Halting the LC-3 ---
```